AEP Agentic-Container OS — Low-Level Design

LLD · v1.1 · Status: Proposed · Branch: `docs/dual-lane-governance-proposal` · Companion: `aep-hld-v1.1.html`
Module path: `github.com/pratik-saptarshi/autonomous-engineering-platform` · Go 1.27.x · stdlib + `golang.org/x/sys` only unless stated

1. Scope and conventions

This document specifies interfaces, data formats, algorithms and an ordered task list for turning the as-built `aep` binary into the dispatching, admission-controlled substrate described in the HLD. Every item is marked AS-BUILT (exists in `develop`) or NEW (to be written). Nothing marked *new* exists yet.

- Process boundary rule: lanes are containers; the `aep` binary on the substrate is the only component that starts, tracks and stops them.
- All persistent state is JSON written atomically (temp file + rename) under `$AEP_STATE_DIR` (default `~/.local/state/aep`; on Windows `%LOCALAPPDATA%\aep`), mode 0700.
- Exit codes: `0` ok · `1` failed check · `2` usage · `3` admission refused · `4` degraded/quarantined · `75` reserved for `ghg` budget denial (passed through unchanged).

2. Repository layout

```
cmd/aep/main.go                      as-built  subcommand switch; flags per subcommand
internal/platform/memory/            as-built  GetMemoryInfo() — linux /proc/meminfo, darwin sysctl, win
internal/platform/lock/              as-built  FileLock — unix.Flock / LockFileEx; TryLock, Lock, Unlock
internal/platform/process/           as-built  ProcessGroup — Setpgid + group signal / Win32 Job Object
internal/verifier/runner/            as-built  DiscoverCommands, RunVerification, TailCappedBuffer, Comm
internal/profile/                    new       hardware profile loading + validation
internal/policy/                     new       policy.data parse + committed-blob hash check
internal/ledger/                     new       beads adapter (issue read/claim/close)
internal/memoryd/                    new       NeuroStrata client (search, inscribe, status)
internal/admission/                  new       memory/CPU reservation accounting
internal/lane/                       new       lane manifest, claim/release/reap, container runtime driv
internal/dispatch/                   new       tier routing + degradation decisions
internal/factory/                    new       thin ESF client (wraps the `factory` CLI)
cmd/aep  (new subcommands)           new       dispatch · lane · profile · boot --factory
deploy/colima/                       new       colima.yaml + substrate compose file
docs/plan/                           this doc  HLD + LLD (html + pdf)
```

3. As-built components (reference)

Package / command	Behaviour that later work depends on
`memory.GetMemoryInfo()`	Returns `MemoryInfo{TotalBytes, AvailableBytes}`. Darwin availability is free + speculative pages only, so it under-reports reclaimable memory; admission must not treat it as exact.
`lock.New(path)`	Advisory exclusive lock. `TryLock()` returns `ErrLocked` if held. The kernel releases it on process death — the property the lane fence relies on.
`process.NewCommand(ctx,name,args,cfg)`	Returns a `ProcessGroup`; `Kill()` terminates the whole tree (group SIGTERM→grace→SIGKILL on POSIX, Job Object on Windows).
`runner.RunVerification(ctx,root,minMem,perCmd,budget)`	Discovers go/cargo/npm/pytest/make checks, runs them sequentially bounded by timeout and budget, stops at first failure, returns `VerificationResult{Pass, Receipts[]}`. Each `CommandReceipt` carries full-stream `StdoutSha` / `StderrSha` plus 64 KiB display tails; the signed `RunReceipt` is finalized by the ledger slice.
`aep boot`	Memory probe, 384 MiB floor, platform line, then `BOOT-GATE-RESULT: SYSTEM-OK`. Does not yet check NeuroStrata, guards, gh auth, binaries or policy.
`aep supervisor [--loop --interval D --once]`	Tick = boot gate then one `RunVerification`. Boot-gate failure is fatal in every mode; verification failure is fatal only outside loop mode. Ticks never overlap.

4. New packages — interfaces

4.1 `internal/profile`

```
type BackendKind string

type IsolationKind uint8
const (
    IsolationNone IsolationKind = iota
    IsolationUserSpaceKernel
    IsolationHardwareMicroVM
)

type Provenance uint8
const (
    ProvenanceLocal Provenance = iota
    ProvenanceRemote
)

type Isolation struct {
    kind     IsolationKind
    evidence [32]byte // probe or attestation digest; zero means unusable
}
func (i Isolation) Valid() bool
func (i Isolation) AtLeast(min IsolationKind) bool // rank only; Valid() checks evidence

type Requirement struct {
    kind IsolationKind
    set  bool
}
func NoFloor() Requirement
func AtLeast(kind IsolationKind) Requirement
func RequirementForRiskClass(class string) (Requirement, error)
func (r Requirement) MetBy(i Isolation) bool // unset requirement fails closed

type Backend struct {
    kind       BackendKind
    isolation  Isolation
    provenance Provenance
}
func NewBackend(kind BackendKind, isolation Isolation, provenance Provenance) (Backend, error)
    // rejects gVisor+hardware, cube-kvm+user-space, remote without attestation,
    // and zero evidence; the zero Backend is not usable.
func (b Backend) Usable() bool
func (b Backend) Isolation() Isolation

type ProbeResult struct {
    Present bool
    Digest  [32]byte
}

type MiB uint64
type Bytes uint64
func (m MiB) Bytes() (Bytes, error) // rejects overflow
type MemoryView struct {
    Total     Bytes // profile minimum
    Available Bytes // admission
```

```
    Limit     Bytes // cgroup or VM limit; zero means none
}

type Profile struct {
    HostFamily      string // "mac-colima-vz" | "linux-native" | "linux-gvisor"; runtime only
    Name            string // e.g. "mac-colima-vz-48g", "linux-native-16g"
    VMCPUs          int    // 2 (nano) up to 12 (workstation)
    VMMemoryMiB     uint64 // 8192 up to 49152
    VMDiskGiB       int    // 30 up to 220
    SubstrateMiB    uint64 // 2560 (nano, 2.5 GiB) up to 8192
    Tier1BudgetMiB  uint64 // 3584 (nano, 3.5 GiB) up to 16384
    Tier2BudgetMiB  uint64 // 1536 (nano, 1.5 GiB) up to 24576
    Tier1LaneMiB    uint64 // 2048 (nano/small/medium) or 1536 (workstation 10-lane preset)
    Tier2LaneMiB    uint64 // 1536 (nano) up to 6144
    MaxTier1Lanes   int    // 1 (nano) up to 10 (workstation; 10 * 1536 = 15360 <= 16384)
    MaxTier2Lanes   int    // 1 (nano) up to 6
    Backend         Backend
    KVMProbe        ProbeResult
    MemFloorMiB     uint64 // 384 (nano) / 512
    ONNXThreads     int
}
func Detect() (Profile, error)            // probes CPU, RAM, disk, /dev/kvm, and container runtime
func Load(path string) (Profile, error)   // KEY=VALUE file, same keys as hardware-profile.conf
func (p Profile) Validate(total memory.Bytes) error // sum(substrate+t1+t2) <= total; ceilings*laneMiB <=
func (p Profile) Tier2Supported() bool { return p.Backend.Usable() }
func (p Profile) Tier2Admissible(laneMiB memory.MiB, active int) error
```

Host-family rule: host family selects the runtime; the resolved `Backend` decides Tier 2. `Tier2Supported()` is true iff `Backend.Usable()`; `KVMProbe` only selects the `cube-kvm` candidate and is never the capability definition. `mac-colima-vz` may resolve `libkrun` without nested KVM; `linux-native` may resolve `gvisor` on KVM-free hosts; `remote-api` counts only when explicitly configured and attested. The nano preset is 2560 + 3584 + 1536 = 7680 MiB (7.5 GiB), with `MaxTier1Lanes=1` and `MaxTier2Lanes=1`. Preset validation must satisfy `MaxTier1Lanes * Tier1LaneMiB <= Tier1BudgetMiB` and the same for Tier 2: nano 1×2048 <= 3584; small 3×2048 <= 6656; medium 6×2048 <= 12288; workstation 10×1536 <= 16384; Tier 2: nano 1×1536 <= 1536, small 1×6144 <= 6144, medium 3×5120 <= 15360, workstation 6×4096 <= 24576. `Detect()` returns the family from `GOOS` + runtime probe, then the size tier from CPU/RAM.

4.2 `internal/policy`

```
type Policy struct { EnvScrubFragments []string; SensitivePaths []string; Raw []byte; SHA256 string }
func Load(path string) (Policy, error)
func (p Policy) VerifyAgainstHEAD(repoDir string) error   // git show HEAD:scripts/policy.data | sha256 =
func (p Policy) ScrubEnv(env []string) []string          // drops names containing any fragment, keeping
func (p Policy) CheckWorkingSet(paths []string) error    // refuses sensitive-path matches
```

Behavioural parity target: `scripts/boot-gate.sh` `check_policy` and `lane-coord.sh`'s working-set check. When `policy.data` is absent, return a distinguishable `ErrPolicyInactive` so boot can print `POLICY-INACTIVE` (exit 0) instead of pretending a check passed.

4.3 `internal/ledger` (beads adapter)

```
type Issue struct { ID string; Title string; Status string; Tier string /* "local"|"factory"|"" */; Prior
type Ledger interface {
    Next(ctx context.Context) (*Issue, error)          // highest-priority open issue with no active lane
    Get(ctx context.Context, id string) (*Issue, error)
    Claim(ctx context.Context, id, owner string) error  // sets in_progress; idempotent for same owner
    Close(ctx context.Context, id, reason string) error
}
func NewBD(binary string) Ledger   // shells out to `bd ... --json`; no direct Dolt access
```

The adapter only parses `bd`'s JSON output. Tier comes from a label (`tier:local` / `tier:factory`); a missing label is treated as `local` only if the issue also carries `risk:low`, otherwise dispatch refuses (exit 4) so untrusted work cannot silently land in the weaker lane.

4.4 `internal/memoryd` (NeuroStrata host relay)

```
type Client interface {
    Status(ctx context.Context) error                                  // maps to `neurostrata-mcp statu
    Search(ctx context.Context, ns, query string, k int) ([]Hit, error)
    Inscribe(ctx context.Context, ns string, rec Record) error
    Guards(ctx context.Context) (n int, hasDestructiveRule bool, err error)
}
// memoryd and gwd implement the shared relay.Handler contract.
type Handler = relay.Handler
func NewCLI(binary string) Client   // wraps the neurostrata-mcp CLI, as boot-gate.sh does today
```

Host supervisor exposes a per-lane UNIX domain socket (`/run/aep/lanes/<laneID>/memory.sock`) mounted into the lane container. The relay intercepts requests, strips credentials, attaches the lane provenance tag, and proxies to NeuroStrata. In host mode the endpoint is `127.0.0.1:34343`; in the compose topology it is `neurostrata:34343` on the private `aep-net` network, where `neurostrata` binds `0.0.0.0:34343` and publishes no host port. A supervisor container cannot use its own loopback as the upstream. MicroVMs use an outbox spool file ingested upon run completion, preserving zero-egress boundaries.

All relays (`memoryd`, `gwd`, `execd`, `gitd`) implement `relay.Handler`; there is no separate `RelayServer` or `GHGRelay` protocol.

4.5 `internal/gateway` (ghg IPC relay)

```
// gwd implements the shared relay.Handler contract; no separate protocol.
type GHGRelay = relay.Handler
type GHGClient interface {
    Watch(ctx context.Context, runID string) error
    API(ctx context.Context, method, endpoint string, body io.Reader) ([]byte, error)
    PRView(ctx context.Context, prNumber int) ([]byte, error)
}
```

Overcomes the fact that `tools/ghg/ghg.py` is a CLI with no network daemon. The host-side `aep gwd` daemon serves an IPC socket (`/run/aep/lanes/<laneID>/ghg.sock`). Inside the lane, a lightweight shim forwards commands to the socket. The host relay executes `ghg.py` locally, enforcing the 3,500 req/h cap and poller coalescing. GitHub PAT credentials never enter the container.

4.6 `internal/admission`

```
type Request struct { LaneID string; Tier int; MemMiB uint64; CPUs float64 }
type Reservation struct { LaneID string; Tier int; MemMiB uint64; CPUs float64; At time.Time }
type Controller struct { /* profile, state file, lock */ }
func (c *Controller) Admit(ctx context.Context, r Request) (Reservation, error)  // ErrNoCapacity or ErrT
func (c *Controller) Release(laneID string) error
func (c *Controller) Snapshot() (Usage, error)
```

State lives in `$AEP_STATE_DIR/admission.json`, guarded by one `lock.FileLock` held only for the read-modify-write. See §6 for the algorithm.

4.7 `internal/lane`

```
type Manifest struct {
    LaneID, IssueID string
    Tier            int
    Image, Container string
    Worktree        string
    ReservedMiB     uint64
    LauncherPID     int
    StartedAt       time.Time
    State           string // claimed|running|verifying|done|failed|reaped
}
type Runtime interface {                      // docker/podman-compatible CLI driver
    Run(ctx context.Context, spec RunSpec) (containerID string, err error)
    Stop(ctx context.Context, id string, grace time.Duration) error
    Inspect(ctx context.Context, id string) (State, error)
}
func Claim(ctx context.Context, d Deps, issue Issue) (Manifest, error)
func Release(ctx context.Context, d Deps, laneID string, clean bool) error
func Reap(ctx context.Context, d Deps) ([]string, error)   // lanes whose launcher is dead AND container
```

The fence is a `lock.FileLock` on `lanes/<id>.fence` held by the launching `aep lane claim --exec` process for the lane's lifetime, mirroring `lane-coord.sh`. A takeover or `release --clean` requires both: fence free *and* recorded container no longer running.

4.8 `internal/dispatch`

```
type Decision struct {
    IssueID string
    Tier    int
    Action  string // "run-local" | "run-factory" | "quarantine" | "fallback-local" | "wait"
    Reason  string
}
func Decide(ctx context.Context, d Deps, issue Issue) Decision
```

Decision table in §7. `Decide` is pure given its probes, so it is unit-testable with fakes.

4.9 `internal/factory`

```
// BackendKind, Isolation, and Requirement are defined in §4.1.
const (
    BackendNone      BackendKind = ""            // no usable backend
    BackendCubeKVM   BackendKind = "cube-kvm"    // Linux with /dev/kvm
    BackendLibkrun   BackendKind = "libkrun"     // macOS Virtualization.framework / vz
    BackendGVisor    BackendKind = "gvisor"      // runsc systrap (no KVM required)
    BackendRemote    BackendKind = "remote-api"  // emergency fallback, attested only
)

func NewBackend(kind BackendKind, isolation Isolation, provenance Provenance) (Backend, error)
    // rejects gVisor+hardware, cube-kvm+user-space, remote without attestation,
    // and zero evidence; the zero Backend is not usable.
func (b Backend) Usable() bool
func (b Backend) Isolation() Isolation

type Client interface {
    Doctor(ctx context.Context) error
    Backend() (Backend, error)
    Run(ctx context.Context, req RunRequest) (RunHandle, error)   // WorkflowID = "esf-beads-" + IssueID
    Status(ctx context.Context, id string) (RunStatus, error)
    PatchPath(ctx context.Context, id string) (string, error)
    VerifyEgressProbe(ctx context.Context, id string) (bool, error)
}
```

Wraps ESF's sandbox provider interface. Selects a local-first backend: native hardware microVMs via `cube-kvm` (Linux) or `libkrun` (macOS `vz`), falling back to `gvisor` (`runsc systrap` user-space kernel) when hardware virtualization is absent. Remote sandbox clusters are strictly an emergency fallback of last resort if all local options fail. A guard reads `factory.toml` and rejects `hardening.acknowledge_open_egress = true` (upstream default is `false`) and ensures closed egress policies. `VerifyEgressProbe` checks the SHA-256 digest of the in-VM probe evidence; a missing or mismatched digest rejects the run.

5. CLI surface (new)

Command	Behaviour	Exits
`aep boot [--factory] [--profile F]`	Extends the as-built gate: binaries (`neurostrata-mcp bd ocr gh`), NeuroStrata status, guard count ≥3 and destructive-actions rule, `gh auth status` , policy hash, memory floor from profile. `--factory` adds `factory doctor` and the egress assertion. Prints the same `BOOT-GATE: <name> ok|FAIL` lines and `BOOT-GATE-RESULT` tokens as the shell gate.	0 / 1
`aep profile show|validate`	Prints the resolved profile and current admission usage.	0 / 1
`aep dispatch [--issue ID] [--dry-run]`	Boot gate → pick issue (or `Next`) → `Decide` → admit → start lane/pipeline. `--dry-run` prints the decision record only.	0 / 3 / 4
`aep lane claim|release|status|reap`	Go port of `lane-coord.sh` 's verbs for container lanes (§4.6).	0 / 1 / 3
`aep supervisor` (extended)	Tick becomes: boot gate → `reap` → dispatch up to the admission ceiling → verification of done lanes. Existing flags unchanged; new `--dispatch` opt-in so current behaviour is preserved by default.	unchanged

6. Admission algorithm

```
Admit(req):
  lock(admission.lock)                          # FileLock.Lock, bounded wait 30s
  s := load(admission.json)
  if req.Tier == Tier2:
    if !profile.Tier2Supported():
      return ErrTier2Unsupported("no local sandbox backend (Cube/KVM, libkrun, or gVisor) available")
    if err := profile.Tier2Admissible(memory.MiB(req.MemMiB), count(tier)); err != nil:
      return err                                  # capacity, not capability

  budget  := profile.TierBudgetMiB(req.Tier)
  used    := sum(s.Reservations[tier == req.Tier].MemMiB)
  ceiling := profile.MaxLanes(req.Tier)
  host    := memory.GetMemoryInfo().AvailableBytes (inside the VM)

  reqBytes, err := memory.MiB(req.MemMiB).Bytes()
  if err != nil || req.MemMiB == 0                  -> ErrInvalidRequest("invalid memory request")
  if count(tier) >= ceiling                         -> ErrNoCapacity("lane ceiling")
  if used > budget || req.MemMiB > budget - used    -> ErrNoCapacity("tier memory budget")
  if host < reqBytes                                -> ErrNoCapacity("host floor")
  if host - reqBytes < profile.MemFloorMiB*MiB      -> ErrNoCapacity("host floor")
  append reservation; write atomically; unlock; return
```

The nano preset is 2560 + 3584 + 1536 = 7680 MiB (7.5 GiB); `MaxTier1Lanes=1` and `MaxTier2Lanes=1` . The subtraction is safe only after the `host < reqBytes` guard; do not reintroduce the unsigned underflow.

- Reservations are keyed by lane ID and removed in `Release` and by `Reap` ; a reservation whose lane manifest is `reaped` is garbage-collected on the next `Admit` .

- The count ceiling and the memory budget are *both* enforced, parameterizing dynamically by the active hardware profile (`nano-8g` to `workstation-48g`).
- Container memory is enforced by the runtime: `--memory=<ReservedMiB>m --memory-swap=<same> --cpus=<n>` , so a lane exceeding its reservation is OOM-killed inside its own cgroup rather than starving the substrate.
- CPU is a soft share (not a hard partition): reservations sum may exceed host cores by design; only memory and sandbox backend availability are admission-blocking resources.

7. Dispatch decision table

Tier tag	Probes	Decision	Exit
`local`	boot ok, admission ok	`run-local`	0
`local`	admission ErrNoCapacity	`wait` (bounded retry, then leave issue open)	3
`factory`	local backend available (Cube/KVM, libkrun, or gVisor) + doctor ok + probe ok + admit ok	`run-factory` (local sandbox)	0
`factory`	all local backends fail + remote cluster configured	`run-remote-factory` (emergency fallback of last resort)	0
`factory`	no local backend available and no remote cluster	`quarantine` : label issue `blocked:unsupported-tier2-no-sandbox`	4
`factory`	Temporal or sandbox daemon unreachable	`quarantine` : label issue `blocked:factory-unavailable` , write decision record, do not start anything	4
`factory`	`acknowledge_open_egress = true` OR probe failed	`quarantine` (governance failure, never auto-fallback)	4
`factory`	unreachable/unsupported *and* operator provides `--override-tier1-risk-reassessment` *and* `fallback:local-ok`	`fallback-local` : run in a fenced Tier-1 lane with `review:escalated` (mandatory human review)	4
missing, no `risk:low`	—	refuse (cannot place untrusted work)	4

Local-First and Quarantine Invariants: Tasks tagged `factory` execute locally using hardware microVMs (`cube-kvm` , `libkrun`) or user-space kernels (`gvisor`). Remote sandbox clusters are strictly an emergency fallback of last resort if all local options fail. Automatic downgrade to Tier 1 is strictly prohibited; downgrade requires an explicit operator risk reassessment override (`--override-tier1-risk-reassessment --reason="..."`).

8. Data formats

8.1 Lane manifest — `$AEP_STATE_DIR/lanes/<laneId>.json`

```
{
  "schema": 1,
  "lane_id": "t1-7f3a9c",
  "issue_id": "aep-142",
  "tier": 1,
  "image": "aep:1.1",
  "container": "aep-lane-t1-7f3a9c",
  "worktree": "/Volumes/Build 1/Git-SCM/worktrees/aep-142",
  "reserved_mib": 2048,
  "launcher_pid": 41233,
  "started_at": "2026-11-08T23:59:00Z",
  "state": "running",
  "policy_sha256": "…",
  "exec_relay": "/run/aep/exec.sock",
  "git_relay": "/run/aep/git.sock",
  "receipts": "/…/aep-142.receipts.jsonl"
}
```

8.2 Dispatch decision record — `$AEP_STATE_DIR/decisions/<ts>-<issue>.json`

```
{ "schema": 1, "issue_id": "aep-142", "tier_tag": "factory",
  "action": "quarantine", "reason": "temporal unreachable",
  "probes": { "boot": "ok", "factory_doctor": "fail: connection refused", "egress": "ok", "admission": "o
  "override": { "used": false, "operator": null, "reason": null },
  "at": "2026-11-08T23:59:00Z" }
```

The decision record is the audit home for the explicit operator override: `override.used`, `override.operator`, and `override.reason` are required when a factory task is downgraded. A lane cannot set its own `MinIsolation` or override; only the authenticated operator path can.

8.3 Identity conventions

- Lane ID: `t<tier>-<6 hex>`. Container name: `aep-lane-<laneId>`. Worktree: `$AEP_WORKTREES_ROOT/<issueId>`.
- ESF workflow ID: `esf-beads-<issueId>`; a second submit for the same issue is rejected by the dispatcher before calling `factory run`.
- Branch: `type/scope-short-description` and Conventional Commits, per repo `AGENTS.md`; commit identity per `AGENTS.md`.

9. Runtime topology

9.1 Container and VM runtimes — by host family

mac-colima-vz (macOS): one large shared Colima VM; Docker/Podman containers are spawned on demand *inside* that VM and bounded by admission control — never one VM per lane. Use `--vm-type vz --mount-type virtiofs`; worktrees are virtiofs-mounted into the VM. Tier 2 hardware microVMs are

supported via Colima nested KVM (requires `--nested-virtualization`, confirmed 2026-11-09) OR natively via `libkrun` using Apple's `Virtualization.framework` (`vz`) without requiring nested KVM. gVisor is Linux-only; if neither nested KVM nor libkrun is available on macOS, Tier-2 tasks quarantine rather than claiming a local gVisor fallback.

linux-native (Linux): no Colima; Podman (rootless) / Docker runs directly on the host under cgroups v2. Tier-2 hardware microVMs run via `cube-kvm` if `/dev/kvm` is present.

linux-gvisor (KVM-Free Linux & Cloud VMs): on hosts lacking `/dev/kvm` (Intel N100, Raspberry Pi 5, or standard non-bare-metal cloud instances), Tier-2 isolation runs locally via gVisor (`runsc` in `systrap` mode). `systrap` intercepts all guest syscalls in user space with zero hardware virtualization requirement, providing a measured kernel boundary on low-power devices.

Remote Sandbox Cluster (Emergency Last Resort): connecting to an external/remote sandbox cluster (e.g. remote Cube/E2B endpoint) is strictly an emergency fallback of last resort if and only if all local isolation backends (Cube/KVM, libkrun, and gVisor) have failed or are impossible.

Confirmed 2026-11-09 on Apple M3 Max (macOS 27.2, Colima 0.10.3, `vz` + `virtiofs`): `colima start --nested-virtualization` exposes `/dev/kvm` inside the VM (and in `--privileged` containers); QEMU `accel=kvm` is accepted for a nested L2 guest; Docker 29.5.2 containers run in-VM; Temporal (sqlite dev server) serves UI on `:8233`. Without `--nested-virtualization`, `/dev/kvm` is absent — the flag is required. CubeSandbox itself was not installed/tested.

```
# macOS (M3 Max / Apple Silicon Workstation):
colima start --profile aep \
  --cpu 12 --memory 48 --disk 220 \
  --vm-type vz --mount-type virtiofs --nested-virtualization

# Linux hosts (Intel N100 / AMD Ryzen / Cloud VMs):
# Runs natively on host cgroups v2 using Podman (rootless) or Docker:
aep boot --profile nano-8g --runtime podman
```

Sandbox backends are probed at boot in order: local KVM -> local libkrun -> local gVisor -> remote fallback. If all options are unavailable, Tier 2 is disabled and untrusted tasks are safely quarantined.

9.2 Substrate compose file (`deploy/compose/substrate.compose.yml`)

```
services:
  neurostrata:   # image bakes neurostrata-mcp + pre-seeded model cache; HF_HUB_OFFLINE=1
    volumes: [ns-data:/data]
    mem_limit: 2g
    expose: ["34343"]
    networks: [aep-net]
    # binds 0.0.0.0:34343 inside aep-net; no host port publish
  aep-supervisor:
    command: ["supervisor","--loop","--dispatch","--interval","2h"]
    depends_on: [neurostrata]
    volumes:
      - aep-state:/state
      - "${AEP_SOCKETS_DIR}:/run/aep"               # host bind, shared with lanes
      - "${AEP_REPO}:${AEP_REPO}:rw"                # same absolute path for linked-worktree gitdir point
      - "${AEP_WORKTREES_ROOT}:${AEP_WORKTREES_ROOT}:rw"
    environment:
      AEP_REPO: "${AEP_REPO}"
      AEP_WORKTREES_ROOT: "${AEP_WORKTREES_ROOT}"
      AEP_SOCKETS_DIR: "${AEP_SOCKETS_DIR}"
      NEUROSTRATA_ENDPOINT: "neurostrata:34343"
      GIT_CONFIG_GLOBAL: /dev/null
      GIT_CONFIG_SYSTEM: /dev/null
      GIT_OPTIONAL_LOCKS: "0"
    mem_limit: 512m
    networks: [aep-net, aep-egress]
volumes: { ns-data: {}, aep-state: {}, aep-sockets: {} }
networks:
  aep-net: { driver: bridge, internal: true }   # NeuroStrata only; no external route
  aep-egress: { driver: bridge }                # ghg.py/GitHub egress for the supervisor only
```

The trusted supervisor joins `aep-egress` so `ghg.py` can reach GitHub; `aep-supervisor` keeps `aep-net` for NeuroStrata. `neurostrata` joins only `aep-net`, and lane containers join neither network (`--network none`). The two-network split keeps memory private while giving the GitHub relay an external route. The supervisor mounts the repository and worktrees at the same absolute host paths, so host-created linked-worktree `.git` pointers resolve inside the supervisor. `git.sock` writes objects/refs there with global/system config disabled and hooks host-controlled; lanes still see no usable `.git`. Relay sockets use a host bind directory (`${AEP_SOCKETS_DIR}`), not a named volume, so the supervisor and the lane bind mount share the same socket files.

The substrate supervisor manages four IPC relays in `aep-sockets` : `memory.sock` (NeuroStrata), `ghg.sock` (GitHub), `exec.sock` (allowlisted command execution), and `git.sock` (host-side status/diff/commit). All four implement one `relay.Handler` contract with peer authentication, framing, size caps, and one audit record; they are not four ad-hoc protocols.

9.3 Lane container run spec

```
docker run -d --name aep-lane-t1-7f3a9c \
  --memory=2048m --memory-swap=2048m --cpus=1 \
  --read-only --tmpfs /tmp:rw,size=256m \
  --network none \                          # no direct network egress
  -v "$AEP_SOCKETS_DIR/lanes/t1-7f3a9c:/run/aep:rw" \ # host bind shared with the supervisor, not a named
  -v "<worktree>:/workspace:rw" \
  -v "$AEP_STATE_DIR/empty-git-file:/workspace/.git:ro" \ # mask the linked-worktree gitdir pointer
  --env AEP_EXEC_SOCKET=/run/aep/exec.sock \
  --env AEP_GIT_SOCKET=/run/aep/git.sock \
  --env-file <scrubbed.env> \               # policy.ScrubEnv; no provider or GitHub tokens
  aep:1.1 lane-exec --issue aep-142
```

> Zero-egress lane architecture.
> The trusted host harness runs the agent loop and all model inference. A lane is a tool executor: it runs argv-level allowlisted commands over `exec.sock`, commits over `git.sock`, and mounts no host `.git` directory (the linked-worktree `.git` file is masked). GitHub PATs, provider keys, and loopback ports never enter the container namespace. Dependency caches and package modules are pre-baked into the lane image or mounted via read-only volume cache. Lane output is untrusted tool data; the host harness never interpolates it into a system prompt.

10. Error handling and failure matrix

Failure	Detection	Behaviour
NeuroStrata down at dispatch	`memoryd.Status`	Boot fails closed (exit 1). Supervisor in loop mode treats it as fatal and lets the service manager own retry cadence.
Tier-2 task when no usable backend	`!profile.Tier2Supported()`	`quarantine`: labeled `blocked:unsupported-tier2-no-sandbox`. Never auto-downgraded to Tier 1.
Tier-2 task when backend exists but capacity is exhausted	`profile.Tier2Admissible`	`quarantine`: labeled `blocked:tier2-capacity-exhausted`; do not wait forever.
Re-verification target cannot meet `MinIsolation`	`Requirement.MetBy`	`quarantine` before the original run; never execute then discover the mismatch.
Egress probe fails or unverified	`VerifyEgressProbe`	`quarantine`: run evidence rejected; PR creation blocked; never merged.
Exec or git relay unavailable	`ECONNREFUSED / EPIPE`	Pause with bounded retry, then `blocked:exec-relay-unavailable` or `blocked:git-relay-unavailable`; never fall back to direct network or a host `.git` mount.
Relay socket communication error	`ECONNREFUSED / EPIPE`	Lane execution pauses; alerts supervisor; never falls back to direct network calls.
Launcher crashes mid-lane	fence free	Lane marked reapable only if the container is also gone; otherwise left running and reported.
Lane exceeds memory reservation	runtime OOM kill (exit 137)	Receipt records failure; issue stays open; reservation released; counts toward 3-strike stop.
Temporal/Cube lost mid-run	`factory status`	Do not retry-submit (workflow ID is idempotency key); mark manifest `failed` and quarantine.
Admission state corrupt	JSON parse error	Refuse to admit (exit 3), preserve file as `.corrupt-<ts>`, require operator `aep lane reap`.
`policy.data` differs from `HEAD`	`VerifyAgainstHEAD`	Boot FAIL; nothing starts.

11. Test plan

Layer	What is tested	How
Unit	`profile` validation & detection, `policy` scrub/hash, `admission` arithmetic, `gateway` relay verbs, `dispatch.Decide` table (every row of §7)	Table-driven Go tests with fakes for ledger, factory, memoryd, gateway, and memory probe.
Concurrency	Two `Admit` calls racing for the last slot; exactly one wins	Goroutines + a real `lock.FileLock` on a temp dir.
Fault injection	Corrupt admission file; stale fence; launcher killed; broken relay socket	Temp state dir; spawned helper process; assert no double admit, no premature reap.
Parity	`aep boot` vs `boot-gate.sh` output tokens for the same stub environment	Reuse shell gate's stub-PATH technique; diff the `BOOT-GATE:` lines.
Integration (needs runtime)	Lane container respects `--memory`; IPC sockets functional with zero direct network	Runs where a container runtime exists; skipped elsewhere with printed reason.
Requirement matrix	`NoFloor` admits Tier 1; unset fails closed; user-space floor rejects hardware-unattested; hardware floor rejects gVisor; exact kind passes	Table-driven `Requirement.MetBy` test, including an unknown risk class.
Relay boundaries	`exec.sock` rejects shell interpolation and unlisted argv; `git.sock` commits on the host and rejects `.git/` paths; lane sees no usable `.git`	Scratch repo + `git -C /workspace rev-parse` must fail; host log must show the commit.
Receipt integrity	`Commands[].StdoutSha` / `StderrSha` hash every byte; nonce/ledger sequence reject replay; signature verifies	128 KiB stream with byte 0 changed; replay after ledger restart; wrong key.
Cross-platform	New packages compile for all four `dist/bin` targets	CI `GOOS/GOARCH` build matrix; runtime-dependent tests gated by build tag or env.

Per repo practice (see `docs/field-learnings.md`), each new test must be shown to fail against a deliberate break of what it guards before it is accepted; a test that cannot fail is not evidence.

12. Implementation plan (PR-sized tasks)

Each task is a separate branch (`type/scope-short-description`), Conventional-Commit titles, one reviewable deliverable. Tasks T1–T5 need no container runtime and can land on any host.

ID	Task	Files	Acceptance
T0 ✅ DONE 2026-11-09	Feasibility spike: KVM / nested virt on macOS (Colima) — confirmed	this doc §9.1 evidence note	`/dev/kvm` exists in-VM with `--nested-virtualization` (M3 Max, Colima 0.10.3); QEMU `accel=kvm` accepted; Docker + Temporal run in-VM. Remaining: CubeSandbox-in-Colima + lane RSS measurement. T9 unblocked; T10 still needs CubeSandbox validation.
T1	`profile` package + `aep profile detect|show`	`internal/profile/*`, `cmd/aep`	Validate rejects over-committed budgets; nano, small, medium, workstation presets load; KVM, libkrun, and gVisor backends detected.
T1b	`gateway` package (`aep gwd` / ghg IPC relay)	`internal/gateway/*`, `cmd/aep`	Socket relay executes `ghg.py`; rate caps and poller coalescing verified without leaking PAT.
T2	`policy` package; HEAD-blob verification	`internal/policy/*`	Matches `boot-gate.sh` behaviour incl. `POLICY-INACTIVE`; tamper test fails gate.
T3	`memoryd` host relay client + `aep boot` full parity	`internal/memoryd/*`, `cmd/aep`	Parity test vs shell gate passes; socket relay forwards to `127.0.0.1:34343` in host mode or `neurostrata:34343` on `aep-net` in compose mode.
T4	`admission` controller	`internal/admission/*`	Race test: one winner for last slot; budget, ceiling, and sandbox backend availability enforced.
T5	`ledger` beads adapter + tier-tag rules	`internal/ledger/*`	Missing-tag policy enforced; `bd` JSON fixtures parsed.
T6	`lane` manifest + fence + Go `claim/release/reap` (runtime driver behind interface, fake in tests)	`internal/lane/*`	Launcher-dead-but-container-alive is refused; socket mounts plumbed.
T7	`dispatch` decision engine + `aep dispatch --dry-run`	`internal/dispatch/*`	Every §7 row covered; quarantine-first on missing local sandbox; decision records written.
T8	Supervisor `--dispatch` opt-in	`cmd/aep`	Default behaviour byte-identical to today; new path behind the flag.
T9	Compose topology + substrate relays + lane image	`deploy/compose/*`, `Containerfile`	Substrate comes up; lane container enforces memory cap and communicates over sockets. *Needs runtime (T0).*
T10	`factory` client + sandbox adapters + egress probe verification + `boot --factory`	`internal/factory/*`	Local backends (Cube/KVM, libkrun, gVisor) supported; open-egress config rejected; probe digest verified; quarantine path exercised; emergency remote fallback connector.

T11	Patch re-verification inside a fresh disposable sandbox meeting `MinIsolation`; `bd close` + NeuroStrata inscription	`internal/factory`, `internal/dispatch`	Untrusted patch re-verified in a fresh sandbox whose isolation meets the task `Requirement`; a gVisor-only host quarantines hardware-floor work before execution.

13. Open items carried from the HLD

- Local sandbox runtime availability across environments (T0) — determines whether Tier 2 runs locally via Cube/KVM, libkrun, or gVisor, or must quarantine (with remote cluster strictly as an emergency fallback of last resort).
- NeuroStrata socket relay wire protocol vs CLI-only integration (§4.4).
- Doc location: `docs/plan/` (this folder) vs the existing `docs/plans/`.
- Independent adversarial review of this LLD before T9–T11.

AEP LLD v1.1 · updated with PR #133 review remediations and multi-hardware profiles on 2026-11-08 · as-built facts verified against `cmd/aep/main.go` in `develop`; everything marked *new* is unimplemented.